ECHO: AN AI-POWERED PERSONAL MEMORY ASSISTANT FOR DOCUMENT UNDERSTANDING AND REMINDER MANAGEMENT

Shreenidhi T
240701503
Department of Computer Science and Engineering
Rajalakshmi Engineering College, Chennai, India
shreenidhi.t.2024.cse@rajalakshmi.edu.in

Abstract

Everyday documents such as receipts, warranty cards, medicine labels and subscription notices contain information that people may need later. ECHO is a web-based personal memory assistant designed to capture, categorize, store and retrieve such information. The prototype combines optical character recognition (OCR), term frequency–inverse document frequency (TF-IDF), supervised text classification using Logistic Regression, SQLite storage and rule-based reminder logic. It is implemented with Python and Flask, with HTML, CSS and JavaScript for the user interface. The current project log reports 378 training samples and 98.7% cross-validation accuracy for the classifier. This result applies to the classification component only; it is not an end-to-end accuracy measure. OCR operation depends on the Tesseract engine being installed and configured. This paper presents the motivation, architecture, implementation, preliminary evaluation, limitations and possible future improvements of ECHO.

1. Introduction

Personal documents often contain dates and details that are useful after the document is first received. For example, a receipt may contain a purchase date, a warranty card may show a coverage period, and a product label may include expiry information. If these details are not organized, they may be difficult to retrieve at the right time. A digital assistant that extracts text from images and organizes it can reduce manual effort and improve access to stored information.

Optical character recognition converts text in images into machine-readable characters. Machine-learning methods can then classify the extracted text into predefined categories. ECHO combines these techniques with a lightweight web interface and database to support personal information management.

2. Problem Statement and Objectives

Important information stored in receipts, medicine labels, warranty cards and other documents may be overlooked or difficult to locate when needed. A convenient system is required to extract text from document images, organize the information, store it for later retrieval and identify relevant dates for reminders.

• Accept document images through a web interface.

• Extract text from supported images using OCR.

• Classify extracted text into categories such as food, warranty, receipt, document, subscription, medicine and other. • Store memory records and related details in a database.

• Apply rules to identify date-based reminders when suitable date information is available.

• Provide a simple interface for viewing memories and reminders.

3. Related Work and Technical Background

3.1 Optical Character Recognition

Tesseract is an open-source OCR engine used to recognize text in images. Smith presents an overview of the Tesseract OCR engine and its recognition approach [1]. OCR results can be affected by image quality, rotation, lighting, layout and font, so extracted text should be checked when correctness is important.

3.2 Text Representation and Classification

TF-IDF represents text using numerical weights that reflect how terms occur in a document and across a collection. A supervised classifier learns associations between these features and labelled categories. Scikit-learn provides implementations of text vectorization and Logistic Regression, together with tools for training and evaluation [2]. In ECHO, TF-IDF is the feature-extraction method and Logistic Regression is the classification algorithm.

3.3 Document Categorization

Previous research has explored OCR and machine-learning methods for categorizing scanned documents. These approaches demonstrate a general workflow in which text is recognized, represented as features and assigned to a category. ECHO applies this general approach to personal documents and does not claim to reproduce the datasets or results of other studies.

4. Proposed System and Architecture

ECHO follows a modular web-application design. The user interacts with a browser interface; Flask routes requests to application logic; OCR and machine-learning modules process the text; SQLite stores structured records; and rule-based logic evaluates available dates for reminders.

• Presentation layer: HTML templates, CSS and JavaScript provide the scan, dashboard, memory, question, reminder and settings pages.

• Application layer: Flask handles HTTP routes, request validation, workflow coordination and responses.

• OCR layer: the Python OCR integration invokes the configured Tesseract engine to convert image content into text.

• Machine-learning layer: TF-IDF converts text into numerical features, and Logistic Regression predicts a category.

• Reminder logic: application rules evaluate available date information and determine which reminders should be shown.

• Persistence layer: SQLite stores structured application records; uploaded images and static assets are handled through the project file structure.

The intended main workflow is: user uploads an image → OCR extracts text → TF-IDF transforms the text → Logistic Regression predicts a category → application logic prepares a memory record and relevant date information → SQLite stores the record → the user reviews memories and reminders through the web interface. The question interface operates through a separate application route, according to the assistant logic implemented in the project.

5. Implementation

The prototype uses Python and Flask for the web application. HTML templates, CSS and JavaScript implement the user interface and client-side interactions. The project separates OCR, text extraction, classification, training and assistant logic into modules. SQLite is used for structured data storage, and sample images are included for demonstrations.

Image capture: the user selects and uploads a supported image.

Text recognition: the OCR integration attempts to extract readable text using Tesseract.

Classification: extracted text is transformed using TF-IDF and passed to the trained Logistic Regression model.

Storage: the application saves the memory and associated fields in SQLite.

Review: users can browse memories and access the reminders page.

Questions: users can submit questions through the web interface and receive responses from the implemented assistant logic.

Environment note: during a recorded test session, Flask started and served the home page and static assets successfully. The application log also reported that Tesseract OCR was not installed or configured. Therefore, a full image-scanning workflow should only be described as verified after the OCR engine is configured and tested.

6. Preliminary Evaluation and Results

The available project log reports that the classifier was trained on 378 samples and achieved 98.7% cross-validation accuracy. This is a preliminary result for the classification component, not for the complete application. It does not measure OCR accuracy, date extraction accuracy or reminder correctness.

A more complete evaluation should include the dataset's class distribution, the validation procedure, per-class precision and recall, macro-F1 score and a confusion matrix. OCR should be tested against manually transcribed images using character error rate or word error rate. Reminder behaviour should be checked against manually verified dates and expected outputs. These additional measurements were not available in the current project log.

7. Feasibility, Applications and Impact

Technical feasibility is supported by Python, Flask and commonly available open-source libraries. SQLite enables local persistence without a separate database server. The use of open-source tools can reduce software costs, although

deployment, maintenance and hardware may still involve costs.

Potential applications include organizing receipts, warranty information, product labels, subscription details and other personal documents. ECHO may reduce repeated manual entry and make saved information easier to find. Its usefulness depends on OCR quality, classification performance, accurate dates and understandable feedback.

8. Limitations and Responsible Use

• OCR can misread text in blurry, rotated, handwritten or poorly lit images.

• The classifier can predict an incorrect category, especially when examples are limited or categories have similar vocabulary.

• Reminders depend on relevant dates being extracted or entered correctly and on the application rules handling them properly.

• The reported cross-validation score does not establish end-to-end reliability.

• Personal documents may contain sensitive information; deployed versions should use access controls, secure file handling, data minimization and deletion options.

• The application should not be the sole source for medicine dosage, medical decisions, legal deadlines or important financial obligations.

9. Future Enhancements

• Install and configure Tesseract reliably and provide clear OCR error handling.

• Improve date and entity extraction with validation and user confirmation.

• Compare additional classifiers using a consistent held-out test set.

• Add multilingual OCR and language-aware text processing.

• Add configurable email, mobile or browser notifications with user permission.

• Strengthen authentication, encryption, privacy controls and secure deployment. • Conduct usability testing and publish reproducible evaluation results.

10. Conclusion

ECHO presents a practical approach to personal document organization by combining OCR, TF-IDF text representation, Logistic Regression classification, SQLite storage and rule-based reminders in a Flask web application. Its modular design separates recognition, categorization, storage and reminder logic. The current project log provides a preliminary classifier cross-validation result, while also showing that OCR configuration is required before image scanning can be validated end to end. Future work should focus on reliable extraction, verified date handling, privacy safeguards and systematic evaluation.

References

R. Smith, “An Overview of the Tesseract OCR Engine,” in Proceedings of the Ninth International Conference on Document Analysis and Recognition (ICDAR), vol. 2, 2007, pp. 629–633. doi: 10.1109/ICDAR.2007.4376991. Available: https://research.google/pubs/an-overview-of-the-tesseract-ocr-engine/

F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011. Available: https://www.jmlr.org/papers/v12/pedregosa11a.html

Flask Documentation. Available: https://flask.palletsprojects.com/

SQLite Documentation. Available: https://www.sqlite.org/docs.html

Python Software Foundation, Python 3 Documentation. Available: https://docs.python.org/3/